A
Minor Project-I Report On

"E-LEARING PORTAL"

Submitted in partial fulfillment of
The requirements for the 5th Semester Sessional Examination of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE & ENGINEERING

By

SIPRA PRIYADARSANI (21UG010619)
MANASWINI PADHI(21UG010420)
SANGHAMITRA PARICHHA(21UG010575)

Under the able Supervision of

Mr.Kedar Nath Panda

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



GIET UNIVERSITY, GUNUPUR

2023 – 24

GIET UNIVERSITY,GUNUPUR

Dist. - Rayagada, Odisha-765022, Contact:- +91 7735745535, 06857-250170,172, Visit us:- **www.giet.edu**

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the project work entitled "E-Learing Protal" is done by Name- Sipra Priyadarsani(21UG010619),Manaswini Padhi(21UG010420),Sanghamitra Parichha(21UG010575) in partial fulfillment of the requirements for the 5th Semester Sessional Examination of Bachelor of Technology in Computer Science and Engineering during the academic year 2023-24. This work is submitted to the department as a part of evaluation of 5th Semester Minor Project-I.

Project Supervisor *Class Teacher*

Project Coordinator,3rd Year *HoD,CSE*

ACKNOWLEDGEMENT

We express our sincere gratitude to Mr. Kedar Nath Panda of Computer Science and Engineering for giving us an opportunity to accomplish the project. Without his active support and guidance , this project report has not been successfully completed.

We also thank Mr. Bhavani Sankar Panda ,Project Coordinator ,Dr. Kakita Murali Gopal ,Head of the Department of Computer Science and Engineering, Prof.(Dr.)Sanjay Kumar Kuanar , Dy. Dean, Computational Science, SOET for their consistent support ,guidance and help.

SIPRA PRIYADARSANI(21UG010619)
MANASWINI PADHI(21UG010420)
SANGHAMITRA PARICHHA(21UG010575)

PROJECT ABSTRACT

The E-Learning portal presented here is a very simple application developed HTML, CSS, Java script, python, Django language. This portal fulfills the thirst of knowledge and offers online content that can be deliver for the learner at any where anytime and any age through a wide range of e-learning solution while compared with traditional learning system. It also provides the rapid access to specific knowledge and information. With the rapid growth of information sources and the time constraint the learning methodology has changed. Learners obtain knowledge through e-learning systems rather than manually teaching and learning. We have also used MYSQL database. Online learning refers to instruction that is deliver electronically through various multimedia and internet platform and applications. E-learning is not just a change technology, it is a part of others redefinition of how we as a species transmit knowledge, skills, and values to younger generation of faculty or student.

With the improvement in the technology and e-learning have played a vital role changing the atmosphere over the learning paradigms. E-learning platform is a web best application that provides a comprehensive e-learning platform for student and teachers.

The platform offers a range of features such as content uploading and downloading, resources sharing, note keeping, dictionary support, Wikipedia, books. The proposed technique is for creating an e-learning system with the implementation of full stack. This e-learning portal is designed to provide an interactive and engaging learning experience of students of all levels. The website is build using HTML, CSS, JavaScript for the frontend, while the backend is developed using python and Django framework.

Starting with IDE, we can use IDE like vs code block. We will using html tags which will used to create the frontend part. We will apply CSS property to make the design frontend part better. Then we use JavaScript for interact with the backend part. Then we use Django using python to add more features.

CHAPTER-1 INTRODUCTION

1.1 PURPOSE

An e-learning portal is a website that offers learners interaction and collaboration on e-learning content like lecture, videos, assignments, You tube, Wikipedia, dictionary. Providing an online platform for student to access educational resources and materials anytime at anywhere. Students can learn without going anywhere from our portal. All the teachers of the courses are upload on the portal so that the student can download them and have it for themselves. One type of e-learning platform is a learning management system. A learning system is based on formalized teaching but with the help of electronic resources is known as E-learning. An e-learning portal accelerates the journey of potential student until enrollment. Overall, an e-learning portal using HTML, CSS, JavaScript, python, Django, and MYSQL can enhance the quality of education by providing a flexible, interactive, and engaging learning environment for students and instructors alike.

1.2 PROJECT SCOPE

The scope of an e-learning portal using HTML, CSS, JavaScript, Python, Django, and MySQL is quite broad. With this combination of technologies, it is possible to create a fully-featured e-learning platform that can support a wide range of learning activity. The scopes are:

I. Lectures to upload assignments and resources for their units.

II. Student to download the resources and upload assignments.

III. It provides an easy-to-use way to manage course websites that include schedule information, announcements, as well as course discussions.

Overall, an e-learning portal developed using HTML, CSS, JavaScript, Python, Django, and MySQL can provide a comprehensive learning experience for users, and can be scaled and customized to meet the need of a wide range of educational institutions and businesses.

1.3 FEATURES

Our project has Some Following Feature:

An e-learning portal built using HTML, CSS, JavaScript, Python, Django, and MySQL can have various Features. Here are some common features that such a portal might have:

I. Content Uploading and Downloading -

The project facilitates upload and downloads to the portal. The content is global and can be accessed by all. Both teacher and students can upload the assignment here.

II. Resource Sharing -

This portal facilitates sharing of resources, teachers can upload notes and give homework to the students.

III. 3.Dictionary-

This portal has a dictionary that helps students to find out the difficult word's meanings. We can also use dictionary to check the spelling of the word. Dictionary may gives other information about words. Dictionary also include grammar and pronunciation.

IV. 4.Youtube-

The portal is also have You tube integrated to it so that students can find relevant videos on particular topics. You tube is a free video sharing website that makes it easy to watch online videos. You can also create and upload your own videos to share with others.

V. Library-

This portal has a huge collection of books which will be useful to the students. Student can access the books and can read the content of the book which available in our library, it is very helpful to the students.

VI. To-Do list -

This portal has a to-do list that allows students to keep track of their work and manage time. Time table of the classes is already mentioned.

VII. Wikipedia-

This portal has in-built Wikipedia which makes it easier to acquire knowledge easily. Basically an Wiki allows non-linear evolving, complex and networked text, while also allowing for editor argument, debate, and interaction regarding the content and formatting.

CHAPTER 2 SYSTEM ANALYSIS

2.1 Software Requirements

i. HTML: HTML, in full hypertext markup language, a formatting system for displaying text, graphics, and audio retrieved over the Internet on a computer monitor. Each retrieval unit is known as a Web page (from World Wide Web) and such pages frequently contain hypertext links that allow related pages to be retrieved. In our project First we will be using the table tag, which will be used to create the structure of the calendar. This will give us an idea of how the calendar is created using HTML. In this section, we will create first the structure of the calendar by using the <table> tag. So this will help us to get the structure of the calendar.

ii. CSS: Cascading Style Sheets, fondly referred to as CSS, is a simply designed language intended to simplify the process of making web pages presentable. CSS allows you to apply styles to web pages. More importantly, CSS enables you to do this independent of the HTML that makes up each web page. We will use some CSS properties and attributes of the table tag to design the calendar. The attributes that we are going to use is the border and cell spacing and cellpadding. Here we have used an interesting property of CSS that is border-collapse.

iii. JavaScript: JavaScript is a lightweight, cross-platform, and interpreted compiled programming language which is also known as the scripting language for webpages. It is well-known for the development of web pages, many non-browser environments also use it. JavaScript can be used for Client-side developments as well as Server-side developments. Then we use java Script to add more features like notes and event reminder system.

iv. MYSQL: MySQL is the de-facto standard database system for web sites with HUGE volumes of both data and end-users (like Facebook, Twitter, and Wikipedia). Another great thing about MySQL is that it can be scaled down to support embedded database applications. MYSQL is an open source relational database management system. For WordPress sites, that means it helps you store all your blog posts, users, plugin information, etc. It stores that information in separate “tables” and connects it with “keys”, which is why it's relational.

v. Django: Django is a high-level Python web framework that enables rapid development of secure and maintainable websites. Built by experienced developers, Django takes care of much of the hassle of web development, so you can focus on writing your app without needing to reinvent the wheel.

vi. Python: In Python web application development, Python is mostly used for building the backend. Python frameworks take care of dealing with tasks that are related to backend application development, such as parsing HTTP requests, generating HTTP responses, accessing the database, authentication, authorization, etc.

vii. Visual Studio Code: Visual Studio Code combines the simplicity of a code editor with what developers need for their core edit-build-debug cycle. It provides comprehensive code editing, navigation, and understanding support along with lightweight debugging, a rich extensibility model, and lightweight integration with existing tools.

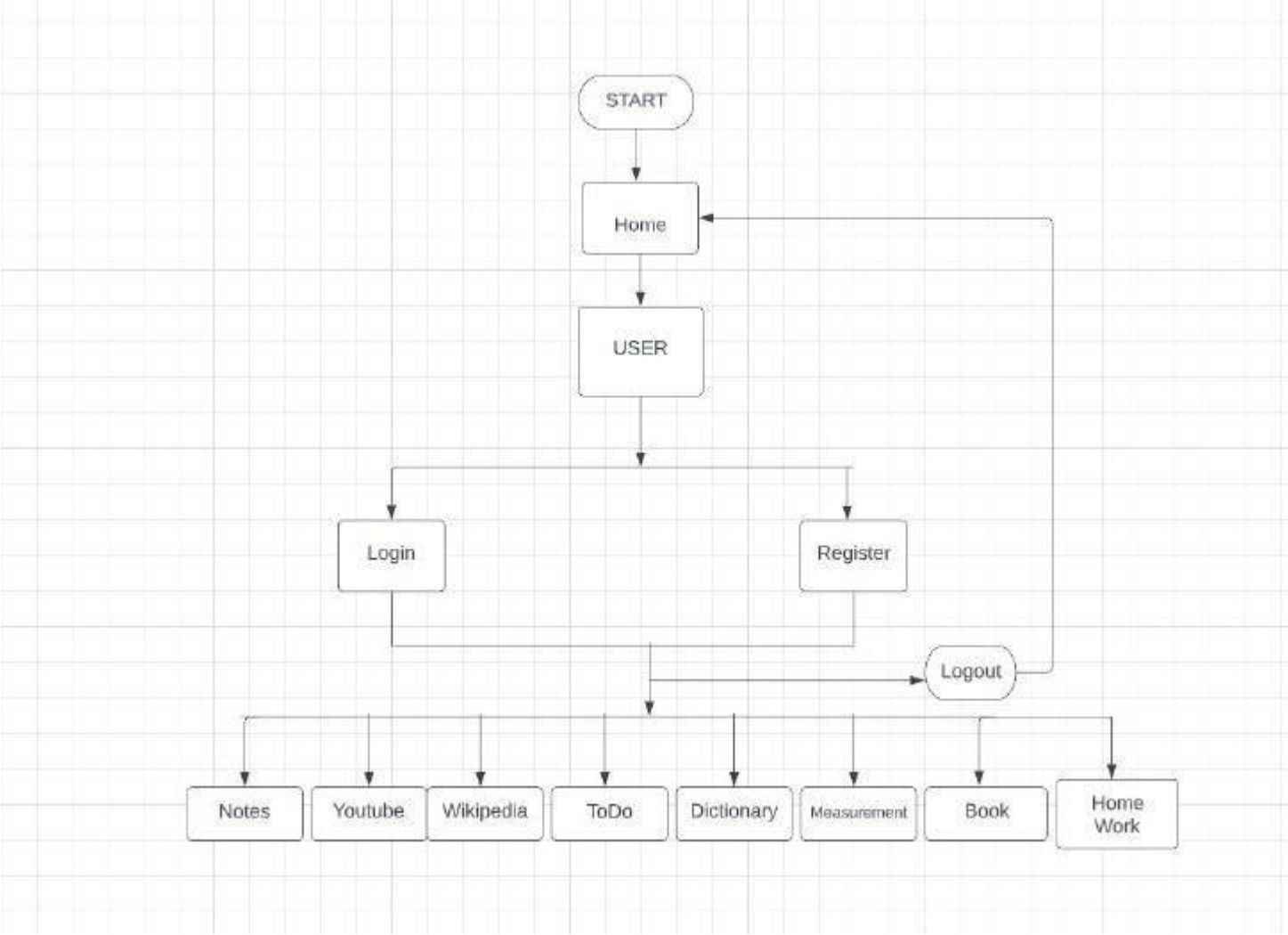
2.2 Hardware Requirements

I. Processor: 900 MHz
II. Memory : 512 MB
III. Disk Space : 750 MB of free space
IV. RAM : 1GB or higher
V. Hard drive : 12GB or more

CHAPTER-3 SYSTEM DESIGN AND SPECIFICATIONS

3.1 HIGH LEVEL DESIGN(HLD)

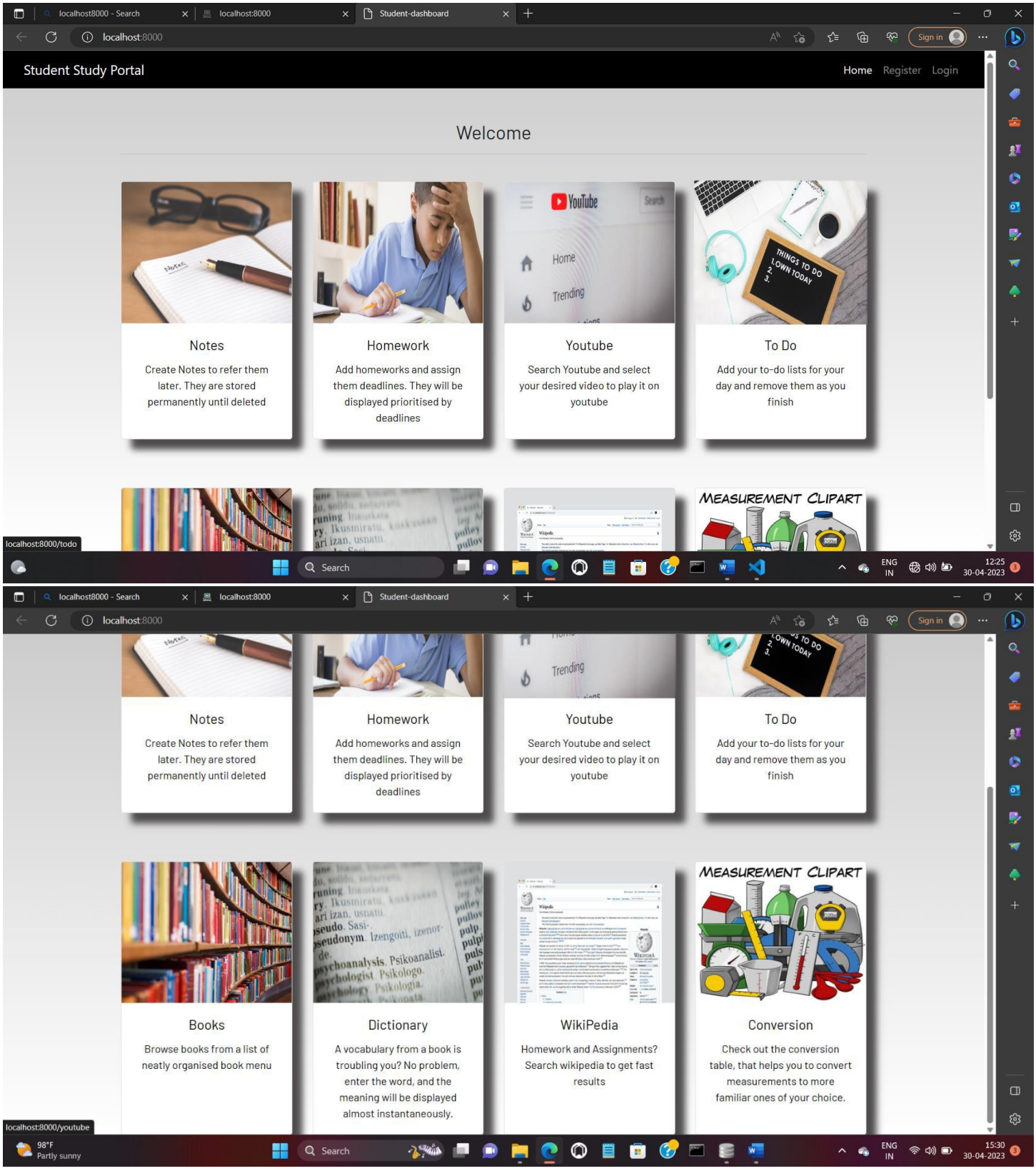
3.1.1 Flow Chart



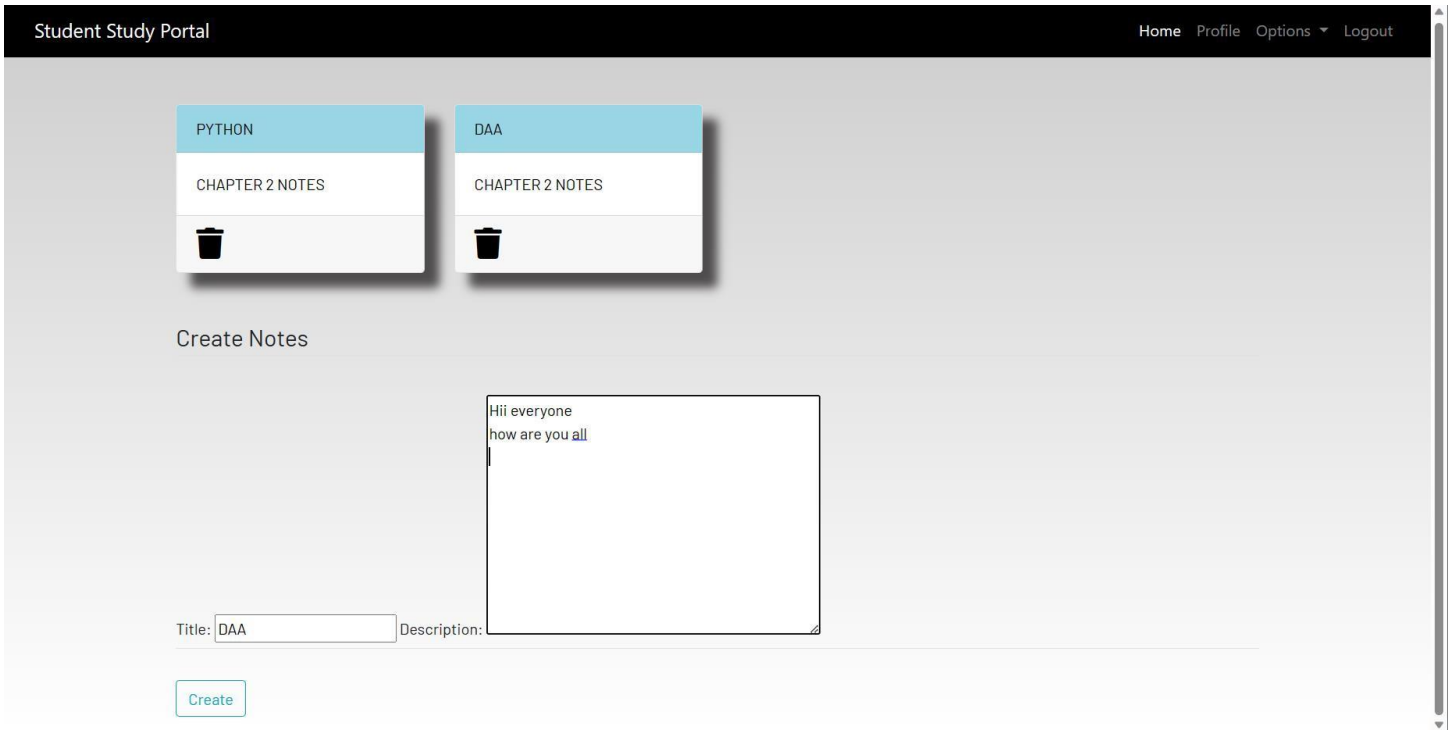
(Project Working Flow Chat)

3.2 LOW LEVEL DESIGN(LLD)

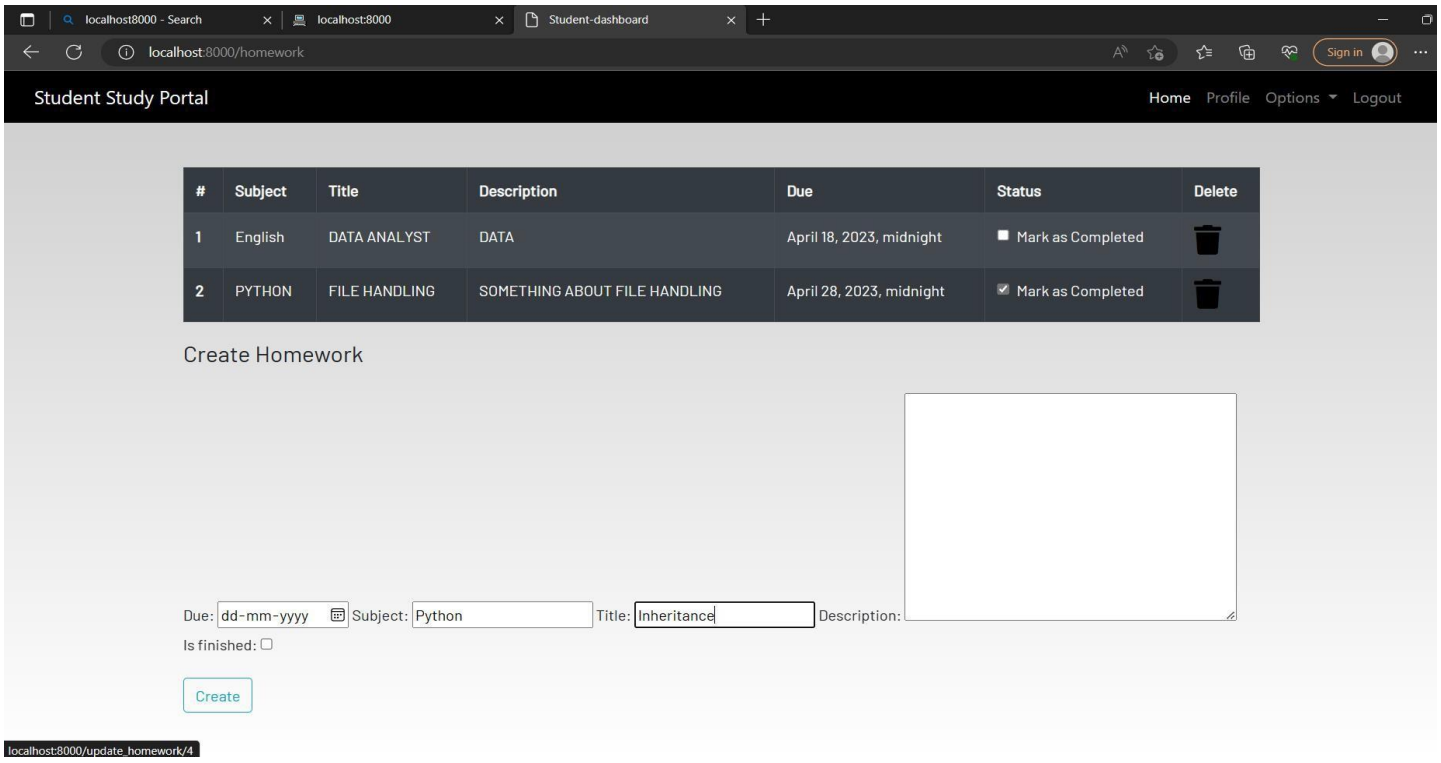
3.2.1 Screen-Shot Diagram



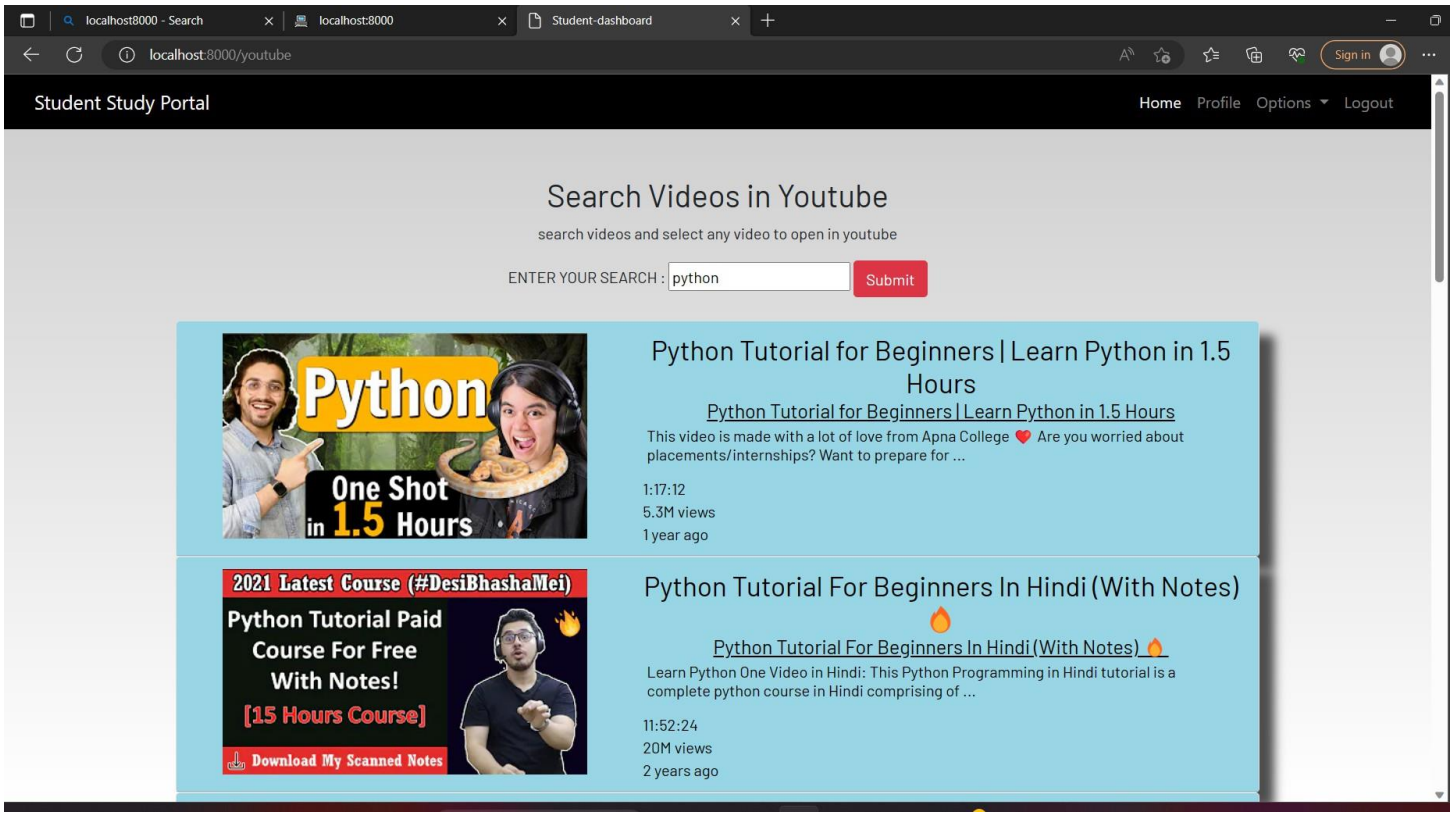
3.2.2 Note



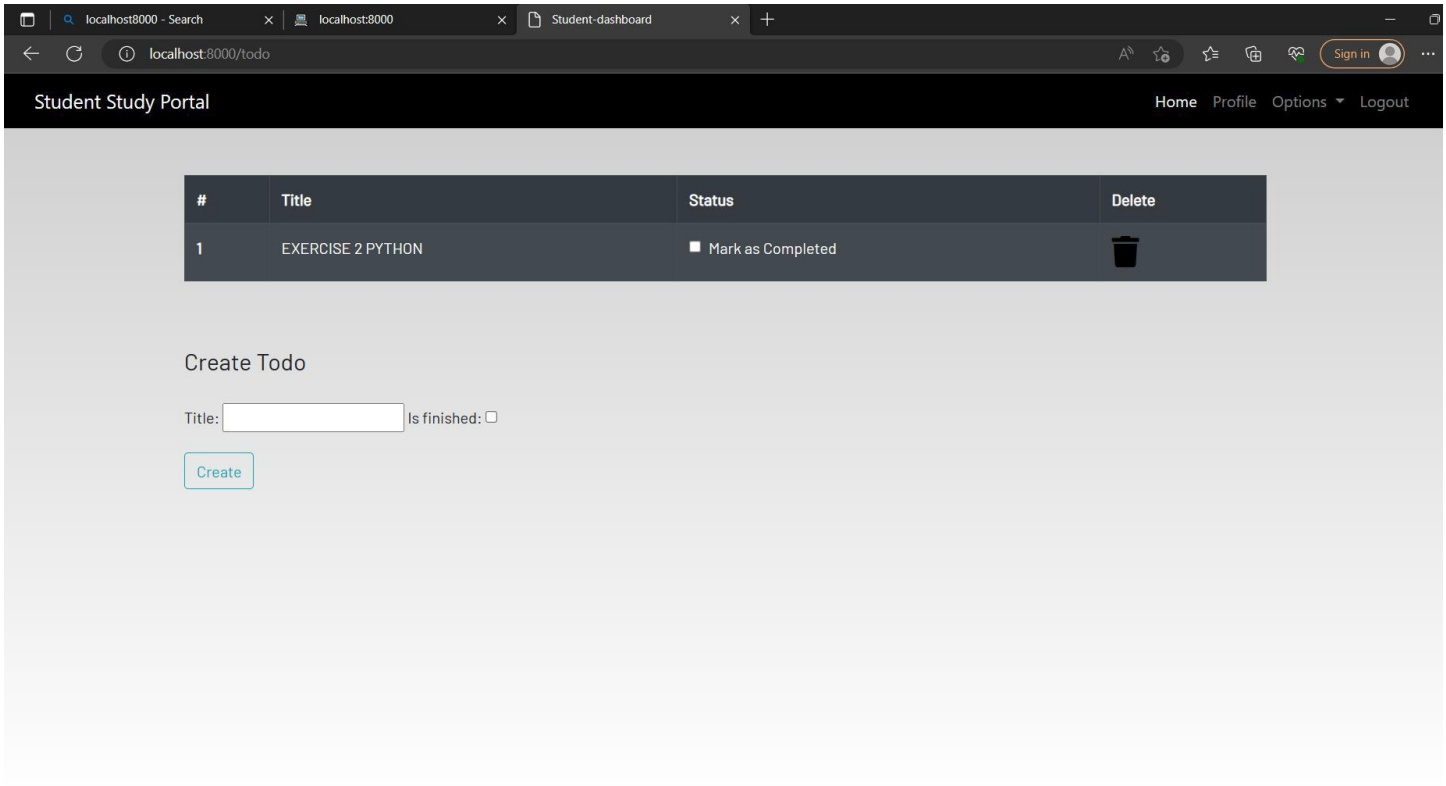
3.2.3 Homework



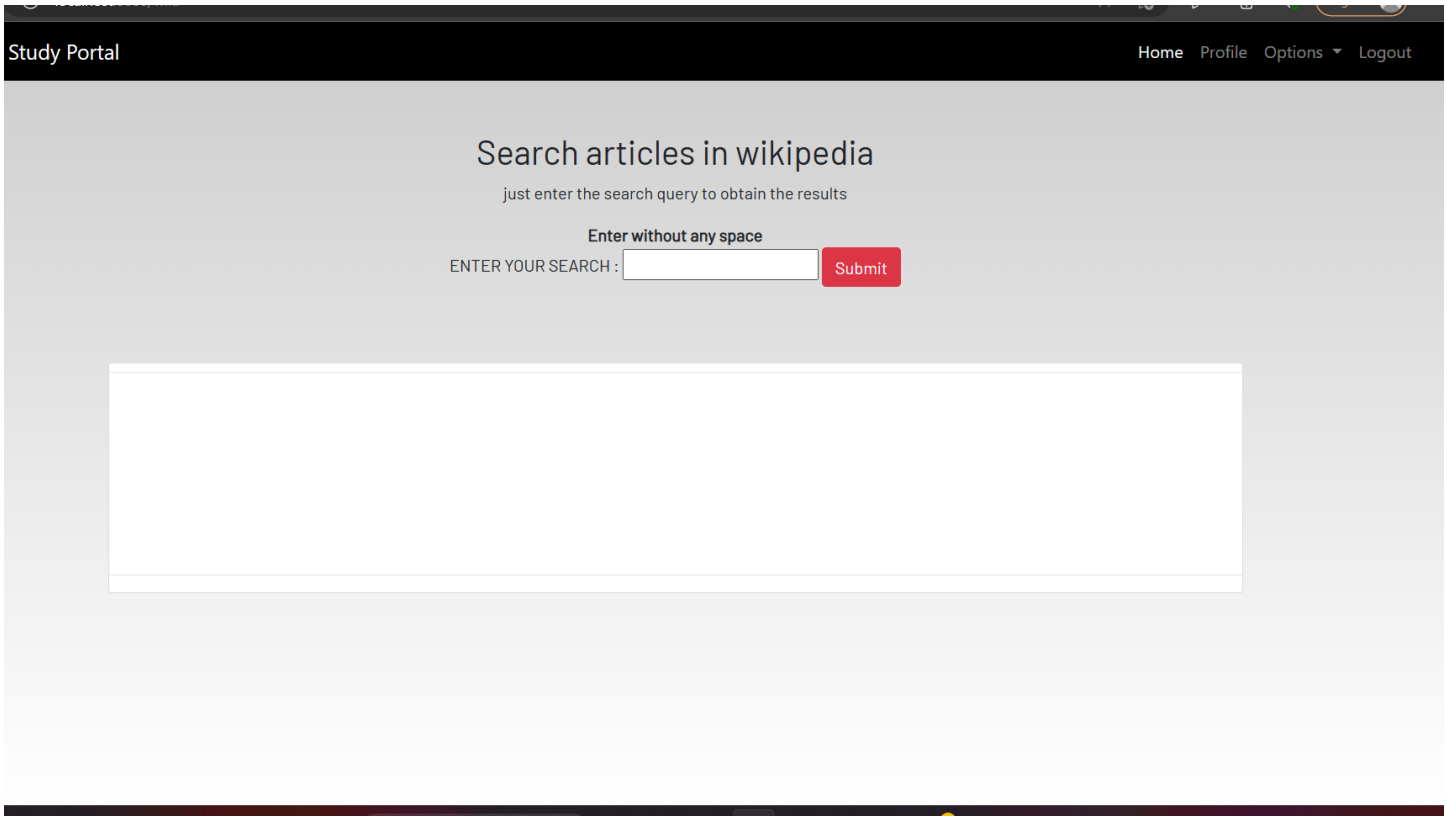
3.2.4 Youtube



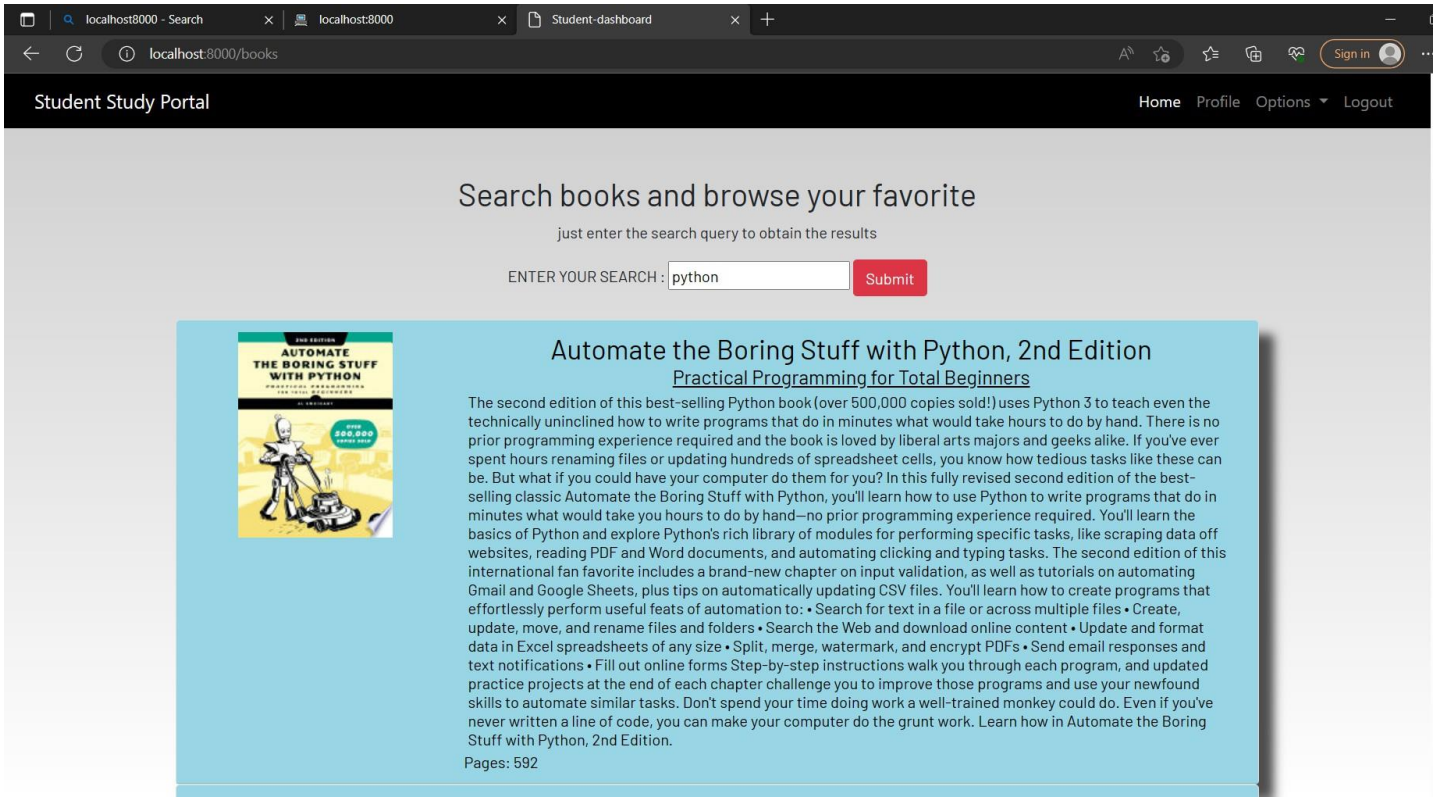
3.2.5 Todo



3.2.6 Wikipedia



3.2.7 Book



CHAPTER-4 CODING

4.1 CODE IN HTML

4.1.1 Login

```
{% load static %}
{% load crispy_forms_tags %}
{% block content %}
<div class="container">
  <form method="POST">
    {% csrf_token %}
    <fieldset class="form-group">
      {{form}}
      <legend class="border-bottom mb-4">Log In</legend>
    </fieldset>
    <div class="form-group">
      <button href="" class="btn btn-outline-info" type="submit">
        Sign In
      </button>
    </div>
  </form>
  <div class="border-top pt-3">
    <small class="text-muted">Need an Account?
      <a class="ml-2" href="{% url 'register' %}">Create an Account</a>
    </small>
  </div>
```

** **

{% endblock content %}

4.1.2 Logout

{% extends 'dashboard/base.html' %}

{% load static %}

{% block content %}

<h2>You are logged Out</h2>

<small class="text-muted"

>Log In Again

</small>

{% endblock content %}

4.1.3 Home

{% extends 'dashboard/base.html' %}

{% load static %}

{% block content %}

<section class="container text-center">

<h3>Welcome</h3>

<hr>

<a href="{% url 'notes' %}">

```
        <img class="card-img-top" src="{% static 'images/notes.jpg' %}" alt="Notes image">
        <div class="card-body">
          <h5 class="card-title ">
            Notes
          </h5>
          Create Notes to refer them later. They are stored permanently until deleted
        </div>
      </div>
    </a>
  </div>
  <div class="col-md-3">
    <a href="{% url 'homework' %}">
      <div class="card">
        <img class="card-img-top" src="{% static 'images/homework.jpg' %}" alt="Notes image">
        <div class="card-body">
          <h5 class="card-title ">
            Homework
          </h5>
          Add homeworks and assign them deadlines. They will be displayed prioritised by deadlines
        </div>
      </div>
    </a>
```

```
    </div>


<div class="col-md-3">
      <a href="{% url 'youtube' %}">
       <div class="card">
         <img class="card-img-top" src="{% static 'images/youtube.jpg' %}" alt="Notes image">
         <div class="card-body">
          <h5 class="card-title ">
            Youtube
          </h5>
          Search Youtube and select your desired video to play it on youtube
         </div>
</div>
      </a>
    </div>
    <div class="col-md-3">
      <a href="{% url 'todo' %}">
       <div class="card">
         <img class="card-img-top" src="{% static 'images/todo.jpg' %}" alt="Notes image">
         <div class="card-body">
    <h5 class="card-title ">
```

```
            To Do


            </h5>
            Add your to-do lists for your day and remove them as you finish
          </div>
        </div>
      </a>
    </div>
  </div>
  <br><br><br>
  <div class="row">
    <div class="col-md-3 ">
      <a href="{% url 'books' %}">
        <div class="card mt-20">
          <img class="card-img-top" src="{% static 'images/books.jpg' %}" alt="Notes image">
          <div class="card-body">

<h5 class="card-title ">


            Books
            </h5>
            Browse books from a list of neatly organised book menu
          </div>
        </div>
```

```
    </a>
  </div>
  <div class="col-md-3">
    <a href="{% url 'dictionary' %}">
      <div class="card">
        <img class="card-img-top" src="{% static 'images/dictionary.jpg' %}" alt="Notes image">
        <div class="card-body">
          <h5 class="card-title ">
            Dictionary
          </h5>
          A vocabulary from a book is troubling you? No problem, enter the word, and the meaning will be displayed
          almost instantaneously.
        </div>
      </div>
    </a>
  </div>
  <div class="col-md-3">
    <a href="{% url 'wiki' %}">
      <div class="card">
        <img class="card-img-top" src="{% static 'images/wiki.jpg' %}" alt="Notes image">
        <div class="card-body">
          <h5 class="card-title ">
            WikiPedia
          </h5>
```

```
        Homework and Assignments? Search wikipedia to get fast results
      </div>
    </div>
  </a>
</div>
<div class="col-md-3">
  <a href="{% url 'conversion' %}">
    <div class="card">
      <img class="card-img-top" src="{% static 'images/conversion.jpg' %}" alt="Notes image">
      <div class="card-body">
        <h5 class="card-title ">
          Conversion
        </h5>
        Check out the conversion table, that helps you to convert measurements to more familiar ones of your choice.
      </div>
    </div>
  </a>
</div>
</div
```

```
</section>

 {% endblock content %}
```

4.1.4 Base

```
 <!DOCTYPE html>

 {% load static %}

 <html lang="en">

<head>

   <meta charset="UTF-8" />

   <meta name="viewport" content="width=device-width, initial-scale=1.0" />

   <meta http-equiv="X-UA-Compatible" content="ie=edge" />

   <title>Student-dashboard</title>

   <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>

   <link rel="stylesheet"
 href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"

     integrity="sha384-
 ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T"
 crossorigin="anonymous" />

   <link rel="stylesheet" href="https://use.fontawesome.com/releases/v5.7.2/css/all.css"

     integrity="sha384-
 fnmOCqbTlWIlj8LyTjo7mOUStjsKC4pOpQbqyi7RrhN7udi9RwhKkMHpvLbHG9Sr"
 crossorigin="anonymous" />

   <link rel="stylesheet" href="{% static 'css/main.css' %}" />

 </head>
```

```
<body>
  <header>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
      <div class="container-fluid">
        <a class="navbar-brand" href="{% url 'home' %}">Student Study Portal</a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent"
          aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">
          <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse justify-content-end" id="navbarSupportedContent">
          <ul class="navbar-nav mb-2 mb-lg-0">
            <li class="nav-item">
              <a class="nav-link active" aria-current="page" href="{% url 'home' %}">Home</a>
            </li>
            {% if user.is_authenticated %}
            <li class="nav-item">
              <a class="nav-link" href="{% url 'profile' %}">Profile</a>
            </li>
            <li class="nav-item dropdown">
              <a class="nav-link dropdown-toggle" href="#" id="navbarDropdown" role="button" data-toggle="dropdown"
                aria-haspopup="true" aria-expanded="false">
                Options
```

```
  </a>
  <div class="dropdown-menu" aria-labelledby="navbarDropdown">
    <a class="dropdown-item" href="{% url 'books' %}">Books</a>
    <a class="dropdown-item" href="{% url 'wiki' %}">Wikipedia</a>
    <div class="dropdown-divider"></div>

    <a class="dropdown-item" href="{% url 'dictionary' %}">Dictionary</a>
    <a class="dropdown-item" href="{% url 'youtube' %}">Youtube</a>
    <div class="dropdown-divider"></div>
    <a class="dropdown-item" href="{% url 'todo' %}">To-do</a>
    <a class="dropdown-item" href="{% url 'homework' %}">Homework</a>
    <div class="dropdown-divider"></div>
    <a class="dropdown-item" href="{% url 'notes' %}">Notes</a>
    <a class="dropdown-item" href="{% url 'conversion' %}">Conversion</a>
    <div class="dropdown-divider"></div>
  </div>
</li>

<li class="nav-item">
  <a class="nav-link" href="{% url 'logout' %}">Logout</a>
</li>
{% else %}
<li class="nav-item">
  <a class="nav-link" href="{% url 'register' %}">Register</a>
```

```
          </li>
          <li class="nav-item">
            <a class="nav-link" href="{% url 'login' %}">Login</a>
          </li>
          {% endif %}
        </ul>
      </div>
    </div>
  </nav>
</header>
<br /><br />
<main>

  {% if messages %}
    {% for msg in messages %}
      <div class ="alert alert-{{msg.tags}}">{{msg}}</div>
    {% endfor %}
  {% endif %}

  {% block content %}
  {% endblock content %}

</main>
<script src="https://cdnjs.cloudflare.com/ajax/libs/jquery/3.3.1/jquery.min.js"></script>
```

```
  <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js"
    integrity="sha384-
UO2eT0CpHqdSJQ6hJty5KVphtPhzWj9WO1clHTMGa3JDZwrnQq4sF86dIHNDz0W1"
crossorigin="anonymous">
  </script>
  <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js"
    integrity="sha384-
JjSmVgyd0p3pXB1rRibZUAYoIIy6OrQ6VrjIEaFf/nJGzIxFDsf4x0xIM+B07jRM"
crossorigin="anonymous">
  </script>
</body>


</html>
```

4.1.5 Profile

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% load crispy_forms_tags%}
{% block content %}
<section class="text-center">
    {% if not todos_done %}
    <div class="container">
        <h2>Due ToDos</h2>
        <table class="table table-striped table-bordered table-hover table-dark">
            <thead>
                <tr>
```

```
        <th scope="col">#</th>
        <th scope="col">Title</th>
        <th scope="col">Status</th>
        <th scope="col">Delete</th>
    </tr>
</thead>
<tbody>
    {%for todo in todos %}
    <tr>
        <th scope="row">{{forloop.counter}}</th>
        <td>{{todo.title}}</td>
        <td>
            <a href="{% url 'update-todo' todo.id %}">
                <div class="form-check">
                    {% if todo.is_finished == True %}
                    <input class="form-check-input" type="checkbox" value="" checked >
                    {% else %}
                    <input class="form-check-input" type="checkbox" value="">
                    {% endif %}
                    <label class="form-check-label text-light" for="flexCheckDefault">
                        Mark as Completed
                    </label>
                </div>
            </a>
```

```
            </td>


            <td><a href="{% url 'delete-todo' todo.id %}"><i class="fa fa-trash fa-
2x"></i></a></td>
        </tr>
        {% endfor %}
      </tbody>
    </table>
  {% else %}
    <h3>All To dos are completed!!!!</h3>
  {% endif %}
    <a href="{% url 'todo' %}" class="btn btn-danger">To Do</a>
    <br><br>


    {% if not homework_done %}
    <h2>Due Homeworks</h2>
    <table class="table table-striped table-bordered table-hover table-dark">
      <thead>
        <tr>
          <th scope="col">#</th>
          <th scope="col">Subject</th>
          <th scope="col">Title</th>
          <th scope="col">Description</th>
          <th scope="col">Due</th>
          <th scope="col">Status</th>
```

```
        <th scope="col">Delete</th>
    </tr>
</thead>
<tbody>
    {% for homework in homeworks %}
    <tr>
        <th scope="row">{{forloop.counter}}</th>
        <td>{{homework.subject}}</td>
        <td>{{homework.title}}</td>
        <td>{{homework.description}}</td>
        <td>{{homework.due}}</td>

        <td>
            <a href="{% url 'update-homework' homework.id %}">
                <div class="form-check">
                    {% if homework.is_finished == True %}
                    <input class="form-check-input" type="checkbox" value="" checked>
                    {% else %}
                    <input class="form-check-input" type="checkbox" value="">
                    {% endif %}
                    <label class="form-check-label text-light" for="flexCheckDefault">
                        Mark as Completed
                    </label>
                </div>
```

```
                    </a>
                </td>
                <td><a href="{% url 'delete-homework' homework.id %}"><i class="fa fa-trash fa-2x"></i></a></td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
    {% else %}
    <h3>All To homeworks are completed!!!!</h3>
    {% endif %}
    <a href="{% url 'homework' %}" class="btn btn-danger">Homeworks</a>
  </div>
</section>

{% endblock content %}
```

4.1.6 Register

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% load crispy_forms_tags %}
{% block content %}

<div class="container">
  <form method="POST">
```

```
    {% csrf_token %}

    <fieldset class="form-group">
      <legend class="border-bottom mb-4">Join Today</legend>
      {{form}}
    </fieldset>
    <div class="form-group">
      <button href="" class="btn btn-outline-info" type="submit">
        Sign Up
      </button>
    </div>
  </form>
  <div class="border-top pt-3">
    <small class="text-muted">Already have an account?
      <a class="ml-2" href="{% url 'login'  %}">Sign In</a>
    </small>
  </div>
</div>
{% endblock content %}
```

4.1.7 Note

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% load crispy_forms_tags %}
```

```
{% block content %}


<div class="container">
  <form method="POST">
    {% csrf_token %}


    <fieldset class="form-group">
      <legend class="border-bottom mb-4">Join Today</legend>
      {{form}}
    </fieldset>
    <div class="form-group">
      <button href="" class="btn btn-outline-info" type="submit">
        Sign Up
      </button>
    </div>
  </form>
  <div class="border-top pt-3">
    <small class="text-muted">Already have an account?
      <a class="ml-2" href="{% url 'login'  %}">Sign In</a>
    </small>
  </div>
</div>
{% endblock content %}
```

4.1.8 Homework

```
{% extends 'dashboard/base.html' %}

{% load static %}

{% load crispy_forms_tags %}

{% block content %}



<div class="container">
   {% if not homework_done %}
   <table class="table table-striped table-bordered table-hover table-dark">
      <thead>
         <tr>
            <th scope="col">#</th>
            <th scope="col">Subject</th>
            <th scope="col">Title</th>
            <th scope="col">Description</th>
            <th scope="col">Due</th>
            <th scope="col">Status</th>
            <th scope="col">Delete</th>
         </tr>
      </thead>
      <tbody>
         {% for homework in homeworks %}
         <tr>
            <th scope="row">{{forloop.counter}}</th>
```

```
            <td>{{homework.subject}}</td>
            <td>{{homework.title}}</td>
            <td>{{homework.description}}</td>
            <td>{{homework.due}}</td>

            <td>
               <a href="{% url 'update-homework' homework.id %}">
                  <div class="form-check">
                     {% if homework.is_finished == True %}
                     <input class="form-check-input" type="checkbox" value="" checked >
                     {% else %}
                     <input class="form-check-input" type="checkbox" value="">
                     {% endif %}
                     <label class="form-check-label text-light" for="flexCheckDefault">
                        Mark as Completed
                     </label>
                  </div>
               </a>
            </td>
            <td><a href="{% url 'delete-homework' homework.id %}"><i class="fa fa-trash fa-2x"></i></a></td>
         </tr>
         {% endfor %}
      </tbody>
   </table>
```

```
    {% else %}
    <h4>All homeworks are completed!! Create to have more!!</h4>
{% endif %}
</div>
<div class=" container">
    <form method="POST">
        {% csrf_token %}
        <fieldset class="form-group">
            <legend class="border-bottom mb-4">Create Homework</legend>
            {{ form }}
        </fieldset>
        <div class="form-group">
            <button href="" class="btn btn-outline-info" type="submit">
                Create
            </button>
        </div>
    </form>
</div>
{% endblock content %}
```

4.1.9 Youtube

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% block content %}
<section class='text-center container'>
```

```
<h2>Search Videos in Youtube</h2>
<p>search videos and select any video to open in youtube</p>
<form action="" method="post">
   {% csrf_token %}
   {{form}}
   <input class="btn btn-danger" type="submit" value="Submit">
</form><br>

{% for result in results %}
<a href="{{result.link}}" target="_blank">
   <div class="card">
      <div class="card-header">
         <div class="row">
            <div class="col-md-5">
               <img class="img-fluid" src="{{result.thumbnail}}" alt="">

            </div>
            <div class="col-md-7">
               <h3 class="p-0 m-0">{{result.title}}</h3>
               <b>
                  <u>
                     <h5 class="p-0 m-0">{{result.channel}}</h5>
                  </u>
               </b>
```

```
                <h6 class="p-0 m-1">{{result.description}}</h6>
                <b>
                    <h6 class="ml-0 mt-3">{{result.duration}}</h6>
                    <h6 class="ml-0 mt-1">{{result.views}}</h6>
                    <h6 class="ml-0 mt-1">{{result.published}}</h6>
                </b>
            </div>
        </div>
    </div>
</div>
</a>
{% endfor %}
<br>
</section>
{% endblock content %}
```

4.1.10 Todo

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% block content %}
<section class='text-center container'>
    <h2>Search Videos in Youtube</h2>
    <p>search videos and select any video to open in youtube</p>
    <form action="" method="post">
        {% csrf_token %}
```

```
    {{form}}
    <input class="btn btn-danger" type="submit" value="Submit">
</form><br>


{% for result in results %}
<a href="{{result.link}}" target="_blank">
    <div class="card">
        <div class="card-header">
            <div class="row">
                <div class="col-md-5">
                    <img class="img-fluid" src="{{result.thumbnail}}" alt="">


                </div>
                <div class="col-md-7">
                    <h3 class="p-0 m-0">{{result.title}}</h3>
                    <b>
                        <u>
                            <h5 class="p-0 m-0">{{result.channel}}</h5>
                        </u>
                    </b>
                    <h6 class="p-0 m-1">{{result.description}}</h6>
                    <b>
                        <h6 class="ml-0 mt-3">{{result.duration}}</h6>
                        <h6 class="ml-0 mt-1">{{result.views}}</h6>
```

```
                    <h6 class="ml-0 mt-1">{{result.published}}</h6>
                </b>
            </div>
        </div>
    </div>
</a>
{% endfor %}
<br>
</section>
{% endblock content %}
```

4.2.1 Book

```
{% extends 'dashboard/base.html' %}
{% load static %}
{% block content %
```

4.3 CODE IN DJANGO

```
.models.deletion.CASCADE,  to=settings.AUTH_USER_MODEL)),
        ],
    ),
]# Generated by Django 4.2 on 2023-04-13 17:50


from django.conf import settings
from django.db import migrations, models
import django.db.models.deletion
```

```
class Migration(migrations.Migration):


    initial = True


    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]

    operations = [
        migrations.CreateModel(
            name='Notes',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
                ('title', models.CharField(max_length=200)),
                ('description', models.TextField()),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE,
to=settings.AUTH_USER_MODEL)),
            ],
        ),
    ]
```

```
# Generated by Django 4.2 on 2023-04-16 19:22

from django.conf import settings
from django.db import migrations, models
import django.db.models.deletion

class Migration(migrations.Migration):

    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
        ('dashboard', '0001_initial'),

    ]

    operations = [
        migrations.AlterModelOptions(
            name='notes',
            options={'verbose_name': 'notes', 'verbose_name_plural': 'notes'},
        ),
        migrations.CreateModel(
            name='HomeWork',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True,
```

```
 serialize=False, verbose_name='ID')),
                ('subject', models.CharField(max_length=50)),
                ('title', models.CharField(max_length=100)),
                ('description', models.TextField()),
                ('due', models.DateTimeField()),
                ('is_finished', models.BooleanField(default=False)),
                ('user', models.ForeignKey(on_delete=django.db.models.deletion.CASCADE,
 to=settings.AUTH_USER_MODEL)),
            ],
        ),
]


    # Generated by Django 4.2 on 2023-04-17 17:40


 from django.conf import settings
 from django.db import migrations, models
```

```
import django.db.models.deletion


class Migration(migrations.Migration):


    dependencies = [
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
        ('dashboard', '0002_alter_notes_options_homework'),
    ]


    operations = [
        migrations.AlterModelOptions(
            name='homework',
            options={'verbose_name': 'homework'},
        ),
        migrations.CreateModel(
            name='Todo',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
                ('title', models.CharField(max_length=100)),
                ('is_finished', models.BooleanField(default=False)),
                ('user', models.ForeignKey(on_delete=django.db
```

4.4 CODE IN PYTHON

4.4.1 Admin

```
from django.contrib import admin
from . models import *

# Register your models here.
admin.site.register(Notes)
admin.site.register(HomeWork)
admin.site.register(Todo)
```

4.4.2 Apps

```
from django.apps import AppConfig

class DashboardConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'dashboard'
```

4.4.3 forms

```
from django import forms
from django.forms import widgets
from . models import *
from django.contrib.auth.forms import UserCreationForm

class NotesForm(forms.ModelForm):
    class Meta:
```

```
        model = Notes
        fields = ['title','description']


class DateInput(forms.DateInput):
    input_type ='date'


class HomeworkForm(forms.ModelForm):
    class Meta:
        model = HomeWork
        widgets = {'due':DateInput()}
        fields = {'subject','title','description','due','is_finished'}


class DashboardFom(forms.Form):
    text = forms.CharField(max_length=100,label="ENTER YOUR SEARCH  ")


class TodoForm(forms.ModelForm):
    class Meta:
        model = Todo
        fields = ['title','is_finished']


class UserRegistrationForm(UserCreationForm):
```

```
    class Meta:
    model = User
        fields = ['username','password1','password2']
```

4.4.4 Models

```
from django.db import models
from django.contrib.auth.models import User

# Create your models here.
class Notes(models.Model):
    user = models.ForeignKey(User,on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    description=models.TextField()

    def __str__(self):
        return self.title

    class Meta:
        verbose_name = "notes"
        verbose_name_plural = "notes"

class HomeWork(models.Model):
    user = models.ForeignKey(User,on_delete=models.CASCADE)
    subject = models.CharField(max_length=50)
```

```
    title = models.CharField(max_length=100)
    description = models.TextField()
    due = models.DateTimeField()
    is_finished=models.BooleanField(default=False)

    class Meta:
        verbose_name = "home works"
        verbose_name = "homework"

    def __str__(self):
        return self.title

class Todo(models.Model):
    user = models.ForeignKey(User,on_delete=models.CASCADE)
    title = models.CharField(max_length=100)
    is_finished = models.BooleanField(default=False)

    def __str__(self):
        return self.title

    class Meta:
        verbose_name = "todo"
        verbose_name = "todo"
```

4.4.5 Urls

```
from django.urls import path
from . import views

urlpatterns = [
    path('',views.home,name="home"),
    path('notes',views.notes,name="notes"),
    path('delete_notes/<int:pk>',views.delete_note,name="delete-note"),
    path('notes_detail/<int:pk>',views.NotesDetailsView.as_view(),name="notes-detail"),


    path('homework',views.homework,name="homework"),
    path('update_homework/<int:pk>',views.update_homework,name="update-homework"),
    path('delete_homework/<int:pk>',views.delete_homework,name="delete-homework"),

    path('youtube',views.youtube,name="youtube"),

    path('todo',views.todo,name="todo"),
    path('update_todo/<int:pk>',views.update_todo,name="update-todo"),
    path('delete_todo/<int:pk>',views.delete_todo,name="delete-todo"),

    path('books',views.books,name="books"),

    path('dictionary',views.dictionary,name="dictionary"),
```

```
    path('wiki',views.wiki,name="wiki"),


    path('conversion',views.conversion,name="conversion"),
    ]
```

4.4.6 views

```
from django import contrib
from django.core.checks import messages
from django.db.models.query import RawQuerySet
from django.forms.widgets import FileInput
from django.shortcuts import redirect,render
from . forms import *
from django.contrib import messages
from django.views import generic
from youtubesearchpython import VideosSearch
import requests
import wikipedia
from django.contrib.auth.decorators import login_required


# Create your views here.
def home(request):
    return render(request,'dashboard/home.html')
@login_required
def notes(request):
```

```
    if request.method == "POST":
        form = NotesForm(request.POST)
        if form.is_valid():
            notes =
 Notes(user=request.user,title=request.POST['title'],description=request.POST['description'])
            notes.save()
        messages.success(request,f"Notes Added from {request.user.username} successfully!")
    else:
        form = NotesForm()
    notes = Notes.objects.filter(user=request.user)
    context = {'notes':notes,'form':form}
    return render(request,'dashboard/notes.html',context)


 def  delete_note(request,pk=None):
    Notes.objects.get(id=pk).delete()
    return redirect("notes")
class NotesDetailsView(generic.DetailView):
models = Notes
 @login_required
 def  homework(request):
    # sourcery skip: assign-if-exp, boolean-if-exp-identity, remove-unnecessary-cast
    if request.method == "POST":
```

```
    form = HomeworkForm(request.POST)
    if form.is_valid():
        try:
            finished =request.POST('is_finished')
            if finished == 'on':
                finished = True
            else:
                finished = False
        except:
            finished = False
        homeworks = HomeWork(
            user = request.user,
            subject = request.POST['subject'],
            title = request.POST['title'],
            description = request.POST['description'],
            due = request.POST['due'],
            is_finished = finished,
        )
        homeworks.save()
        messages.success(request,f'Homework Added from {request.user.username}')
else:
        form = HomeworkForm()
homework = HomeWork.objects.filter(user=request.user)
if len(homework) == 0:
```

```
        homework_done =True
    else:
        homework_done =False


    context = {
        'homeworks':homework,
        'homeworks_done':homework_done,
        'form':form,
    }


    return render(request,'dashboard/homework.html',context)
@login_required
def update_homework(request,pk=None):
    # sourcery skip: assign-if-exp, boolean-if-exp-identity, remove-unnecessary-cast
    homework = HomeWork.objects.get(id=pk)
    if homework.is_finished == True:
        homework.is_finished = False
    else:
        homework.is_finished = True
    homework.save()
    return redirect('homework')
```

```
def delete_homework(request,pk=None):
    HomeWork.objects.get(id=pk).delete()
    return redirect("homework")


def youtube(request):
    # sourcery skip: extract-method, merge-dict-assign, move-assign-in-block
    if request.method == "POST":
        form = DashboardFom(request.POST)
        text = request.POST['text']
        video = VideosSearch(text,limit=10)
        result_list = []
        for i in video.result()['result']:
            result_dict = {
                'input':text,
                'title':i['title'],
                'duration':i['duration'],
                'thumbnail':i['thumbnails'][0]['url'],
                'channel':i['title'],
                'link':i['link'],
                'views':i['viewCount']['short'],
                'published':i['publishedTime'],
            }
            desc = ''
            if i['descriptionSnippet']:
```

```
            for j in i['descriptionSnippet']:
                desc += j['text']
            result_dict['description'] = desc
            result_list.append(result_dict)
            context={
                'form':form,
                'results':result_list

            }
        return render(request,'dashboard/youtube.html',context)
    else:
        form = DashboardFom()
    context = {'form':form}
    return render(request,"dashboard/youtube.html",context)
@login_required
def todo(request):
    # sourcery skip: assign-if-exp, boolean-if-exp-identity, remove-unnecessary-cast
    if request.method == 'POST':
        form = TodoForm(request.POST)
        if form.is_valid():
            try:
                finished = request.POST("is_finished")
                if finished == 'on':
                    finished = True
```

```
            else:
                finished = False
        except:
            finished = False
        todos = Todo(
            user = request.user,
            title = request.POST['title'],
            is_finished = finished


        )
        todos.save()
        messages.success(request,f"TODO ADDED FROM {request.user.username}!!")
else:
    form = TodoForm()
todo = Todo.objects.filter(user=request.user)
if len(todo) == 0:
    todos_done = True
else:
    todos_done = False


context ={
    'form': form,
    'todos':todo,
    'todos_done':todos_done
```

```
    }
    return render(request,"dashboard/todo.html",context)
@login_required
def update_todo(request,pk=None):
    # sourcery skip: assign-if-exp, boolean-if-exp-identity, remove-unnecessary-cast
    todo = Todo.objects.get(id=pk)
    if todo.is_finished == True:
        todo.is_finished = False
    else:
        todo.is_finished =True
    todo.save()
    return redirect('todo')

def delete_todo(request,pk=None):
    Todo.objects.get(id=pk).delete()
    return redirect("todo")

def books(request):
    # sourcery skip: extract-method, merge-dict-assign, move-assign-in-block, use-fstring-for-concatenation
    if request.method == "POST":
        form = DashboardFom(request.POST)
        text = request.POST['text']
        url ="https://www.googleapis.com/books/v1/volumes/?q="+text
```

```
    r = requests.get(url)
    answer = r.json()
    result_list = []
    for i in range(10):
        result_dict = {
            'title':answer['items'][i]['volumeInfo']['title'],
            'subtitle':answer['items'][i]['volumeInfo'].get('subtitle'),
            'description':answer['items'][i]['volumeInfo'].get('description'),
            'count':answer['items'][i]['volumeInfo'].get('pageCount'),
            'catagories':answer['items'][i]['volumeInfo'].get('catagories'),
            'rating':answer['items'][i]['volumeInfo'].get('pageRating'),
            'thumbnail':answer['items'][i]['volumeInfo'].get('imageLinks').get('thumbnail'),
            'preview':answer['items'][i]['volumeInfo'].get('previewLinks'),

        }
        result_list.append(result_dict)
        context={
                'form':form,
                'results':result_list

        }
    return render(request,'dashboard/books.html',context)
else:
    form = DashboardFom()
```

```
        context = {'form':form}
    return render(request,"dashboard/books.html",context)


def dictionary(request):
    # sourcery skip: extract-method, inline-immediately-returned-variable, remove-unnecessary-else, use-fstring-for-concatenation
    if request.method == "POST":
        form = DashboardFom(request.POST)
        text = request.POST['text']
        url ="https://api.dictionaryapi.dev/api/v2/entries/en_US/"+text
        r = requests.get(url)
        answer = r.json()
        try:
            phonetics = answer[0]['phonetics'][0]['text']
            audio = answer[0]['phonetics'][0]['audio']
            definition = answer[0]['meanings'][0]['definitions'][0]['definition']
            example = answer[0]['meanings'][0]['definitions'][0]['example']
            context={
                    'form':form,
                    'input':text,
                    'phonetics':phonetics,
                    'audio':audio,
                    'definition':definition,
                    'example':example,
```

```
            }
        except:
            context = {
                'form':form,
                'input':''
            }
        return render(request,"dashboard/dictionary.html",context)
    else:
        form = DashboardFom()
        context = {'form':form}
    return render(request,"dashboard/dictionary.html",context)


def wiki(request):
    if request.method == "POST":
        text = request.POST['text']
        form = DashboardFom(request.POST)
        search = wikipedia.page(text)
        context ={
            'form':form,
            'title':search.title,
            'link':search.url,
            'details':search.summary,
        }
        return render(request,"dashboard/wiki.html",context)
```

```
    else:
        form = DashboardFom()
    context = {'form':form}
    return render(request,"dashboard/wiki.html",context)
}
```

Chapter-5 Testing

Manual Testing is a type of Software Testing where we manually execute test cases without using any automation tools. The key concept of manual testing is to ensure that the application is error free and it is working in conformance to the specified functional requirements. Test Suites or cases, are designed during the testing phase and should have 100% test coverage. Manual Testing requires more effort but is necessary to check automation feasibility. Manual Testing does not require knowledge of any testing tool. One of the Software Testing Fundamental is "100% Automation is not possible ". This makes Manual Testing imperative.

CHAPTER-6 CONCLUSION & LIMITATION

6.1 CONCLUSION

Ed-hub is a comprehensive e-learning platform that provides a range of features for students and teachers. Ed-hub is an excellent example of how modern web technologies can be used to build robust and user-friendly e-learning platforms. The E-learning Portal is a comprehensive and innovative solution that addresses the need for accessible, affordable, and effective education. Ed hub is a comprehensive e-learning platform that provides a range of features for students and teachers. E learning is an excellent example of how modern web technologies can be used to build robust and user-friendly e-learning platform. The E-learning Portal is a comprehensive and innovative solution that addresses the need for accessible, affordable, and effective education.

6.2 LIMITATION

Although we have put our best efforts to make the website flexible, easy to operate but limitations cannot be ruled out even by us. it was not possible to make the software full proof and dynamic. Lack of time also compelled us to ignore some part.

List of limitations which are present in Our Calendar Application Project:

1. A lack of face to face communication with the instructor inhibits student feedback, causes social isolation, and could cause students to face lack of pressure.
2. Our project doesn't support video calling feature between the faculty and student.
3. It can't be very useful for big academic, because our project has a limited capacity for user login.
4. A lack of pressure is a disadvantage because it causes students to abandon their studies more easily.

Chapter-7 Reference/Bibliography

- https://www.youtube.com/
- https://www.google.com/